

```

- news_service
- joke_service

Examples:
Query: What is the weather forecast for today?
Category: weather_service

Query: What's the latest news?
Category: news_service

Query: Tell me a joke.
Category: joke_service

Only return the category name.

Query: {state.input_data}
"""

classification = llm.invoke([SystemMessage(content=prompt)]).strip()
category = classification.split()[-1]
log_event("Super Agent", f"Classified as: {category}")
return ExpertSystemState(input_data=state.input_data,
category=category)

def weather_service(state: ExpertSystemState) ->
ExpertSystemState:
    log_event("Weather Service Agent", f"Providing weather
forecast for: {state.input_data}")

    prompt = f"Provide the weather forecast for: {state.
input_data}"
    response = llm.invoke([SystemMessage(content=prompt)]).
strip()

    log_event("Weather Service Agent", f"Generated Response:
{response}")
    return ExpertSystemState(input_data=state.input_data,
category=state.category, response=response)

def news_service(state: ExpertSystemState) ->
ExpertSystemState:
    log_event("News Service Agent", f"Providing latest news
for: {state.input_data}")

    prompt = f"Provide the latest news for: {state.input_
data}"
    response = llm.invoke([SystemMessage(content=prompt)]).
strip()

    log_event("News Service Agent", f"Generated Response:
{response}")

```

```

    return ExpertSystemState(input_data=state.input_data,
category=state.category, response=response)

def joke_service(state: ExpertSystemState) ->
ExpertSystemState:
    log_event("Joke Service Agent", f"Telling a joke for:
{state.input_data}")

    prompt = f"Tell a joke for: {state.input_data}"
    response = llm.invoke([SystemMessage(content=prompt)]).
strip()

    log_event("Joke Service Agent", f"Generated Response:
{response}")
    return ExpertSystemState(input_data=state.input_data,
category=state.category, response=response)

```

The `StateGraph` is created to build the workflow, which defines the flow of execution. Each service agent is added as a 'node' in the graph.

```

workflow = StateGraph(state_schema=ExpertSystemState)
workflow.add_node("super_agent", super_agent)
workflow.add_node("weather_service", weather_service)
workflow.add_node("news_service", news_service)
workflow.add_node("joke_service", joke_service)

```

The *decision_function* decides which service to route the query to, based on the category (`weather_service`, `news_service`, `joke_service`). If the category is not found, it defaults to the weather service.

```

def decision_function(state: ExpertSystemState):
    routing = {
        "weather_service": "weather_service",
        "news_service": "news_service",
        "joke_service": "joke_service"
    }
    return routing.get(state.category, "weather_service")

```

```

workflow.add_conditional_edges("super_agent", decision_
function)

```

To set workflow entry and exit points, use the following code:

```

workflow.set_entry_point("super_agent") # Start point
workflow.set_finish_point("weather_service") # Possible
endpoint
workflow.set_finish_point("news_service") # Possible endpoint
workflow.set_finish_point("joke_service") # Possible endpoint

```